

Informatica® Data Replication
9.8.0

Scripting Guide

Informatica Data Replication Scripting Guide
9.8.0
December 2018

© Copyright Informatica LLC 2011, 2019

This software and documentation are provided only under a separate license agreement containing restrictions on use and disclosure. No part of this document may be reproduced or transmitted in any form, by any means (electronic, photocopying, recording or otherwise) without prior consent of Informatica LLC.

U.S. GOVERNMENT RIGHTS Programs, software, databases, and related documentation and technical data delivered to U.S. Government customers are "commercial computer software" or "commercial technical data" pursuant to the applicable Federal Acquisition Regulation and agency-specific supplemental regulations. As such, the use, duplication, disclosure, modification, and adaptation is subject to the restrictions and license terms set forth in the applicable Government contract, and, to the extent applicable by the terms of the Government contract, the additional rights set forth in FAR 52.227-19, Commercial Computer Software License.

Informatica and the Informatica logo are trademarks or registered trademarks of Informatica LLC in the United States and many jurisdictions throughout the world. A current list of Informatica trademarks is available on the web at https://www.informatica.com/trademarks.html. Other company and product names may be trade names or trademarks of their respective owners.

Portions of this software and/or documentation are subject to copyright held by third parties. Required third party notices are included with the product.

The information in this documentation is subject to change without notice. If you find any problems in this documentation, report them to us at infa_documentation@informatica.com.

Informatica products are warranted according to the terms and conditions of the agreements under which they are provided. INFORMATICA PROVIDES THE INFORMATION IN THIS DOCUMENT "AS IS" WITHOUT WARRANTY OF ANY KIND, EXPRESS OR IMPLIED, INCLUDING WITHOUT ANY WARRANTIES OF MERCHANTABILITY, FITNESS FOR A PARTICULAR PURPOSE AND ANY WARRANTY OR CONDITION OF NON-INFRINGEMENT.

Publication Date: 2019-03-26

Table of Contents

Preface 5

Informatica Resources 5

- Informatica Network 5
- Informatica Knowledge Base 5
- Informatica Documentation 5
- Informatica Product Availability Matrixes 6
- Informatica Velocity 6
- Informatica Marketplace 6
- Informatica Global Customer Support 6

Chapter 1: Scripting Overview 7

Using Tcl Scripts and SQL Expressions in Data Replication 7

Adding Tcl and SQL Expressions 8

- Adding Tcl Expressions 8
- Adding SQL Expressions 9
- Adding a Virtual Column with an SQL or Tcl Expression 10

Defining a Virtual Index for the SQL Script Engine and Tcl Script Engine Use 12

Chapter 2: Tcl Scripting 14

Using Tcl Scripts 14

- Tcl Script Execution 14
- Accessing Input Data 14
- Setting the Virtual Column Value 15
- Tcl Script Engine Processing Stages 15
- Scripting Considerations 16
- Global Context 17
- Script Execution Control 18

::dbs:: Tcl Namespace Reference 18

- I/O Utility Commands 18
- Datatype Object Commands 19
- Tcl Script Engine Interaction Commands 21
- Data Manipulation Commands 22
- Change Metadata and Applier State Commands and the ::dbs::repo Variable 23
- SQL Execution Commands 25

Examples of Tcl Scripts 26

- Getting Started with Some Simple Scripts 26
- Filtering Change Records 29
- Handling Undo and Redo Values in Merge Apply Processing 29
- Adding Tcl Packages and Third-Party Extensions to a Script 30
- Using Tcl Scripts to Meet Basic Transformation Requirements 30

Chapter 3: SQL Scripting. 32
Using SQL Expressions. 32
Execution of SQL Expressions. 32
Generating Audit Log Tables for Processing SQL Expressions. 33
Syntax of SQL Expressions. 34
Getting Access to Transactional Data. 34
WHERE Clauses in SQL Expressions. 34
Validating SQL Expressions. 34
Data Access Operators. 35
Access to REDO Values. 35
Access to UNDO Values. 36
Access to Metadata. 37
SQL Script Examples. 39
String Extraction Example. 39
String Concatenation Example. 40
Lookup Example. 41
String Substitution Example. 41
Date Manipulation Example. 42
Computing Example. 42
Current Timestamp Example. 43

Index. 44

Preface

This *Informatica Data Replication Scripting Guide* is written for the developers, software engineers, and database administrators responsible for creating Informatica® Data Replication configurations. This guide assumes that you are familiar with relational database concepts, the database engines, Tcl scripting language, and Structured Query Language (SQL) of the target database.

Informatica Resources

Informatica Network

Informatica Network hosts Informatica Global Customer Support, the Informatica Knowledge Base, and other product resources. To access Informatica Network, visit https://network.informatica.com.

As a member, you can:

- Access all of your Informatica resources in one place.
- Search the Knowledge Base for product resources, including documentation, FAQs, and best practices.
- View product availability information.
- Review your support cases.
- Find your local Informatica User Group Network and collaborate with your peers.

Informatica Knowledge Base

Use the Informatica Knowledge Base to search Informatica Network for product resources such as documentation, how-to articles, best practices, and PAMs.

To access the Knowledge Base, visit https://kb.informatica.com. If you have questions, comments, or ideas about the Knowledge Base, contact the Informatica Knowledge Base team at KB_Feedback@informatica.com.

Informatica Documentation

To get the latest documentation for your product, browse the Informatica Knowledge Base at https://kb.informatica.com/_layouts/ProductDocumentation/Page/ProductDocumentSearch.aspx.

If you have questions, comments, or ideas about this documentation, contact the Informatica Documentation team through email at infa_documentation@informatica.com.

Informatica Product Availability Matrixes

Product Availability Matrixes (PAMs) indicate the versions of operating systems, databases, and other types of data sources and targets that a product release supports. If you are an Informatica Network member, you can access PAMs at https://network.informatica.com/community/informatica-network/product-availability-matrices.

Informatica Velocity

Informatica Velocity is a collection of tips and best practices developed by Informatica Professional Services. Developed from the real-world experience of hundreds of data management projects, Informatica Velocity represents the collective knowledge of our consultants who have worked with organizations from around the world to plan, develop, deploy, and maintain successful data management solutions.

If you are an Informatica Network member, you can access Informatica Velocity resources at http://velocity.informatica.com.

If you have questions, comments, or ideas about Informatica Velocity, contact Informatica Professional Services at ips@informatica.com.

Informatica Marketplace

The Informatica Marketplace is a forum where you can find solutions that augment, extend, or enhance your Informatica implementations. By leveraging any of the hundreds of solutions from Informatica developers and partners, you can improve your productivity and speed up time to implementation on your projects. You can access Informatica Marketplace at https://marketplace.informatica.com.

Informatica Global Customer Support

You can contact a Global Support Center by telephone or through Online Support on Informatica Network.

To find your local Informatica Global Customer Support telephone number, visit the Informatica website at the following link:
http://www.informatica.com/us/services-and-training/support-services/global-support-centers.

If you are an Informatica Network member, you can use Online Support at http://network.informatica.com.

CHAPTER 1

Scripting Overview

This chapter includes the following topics:

- Using Tcl Scripts and SQL Expressions in Data Replication, 7
- Adding Tcl and SQL Expressions, 8
- Defining a Virtual Index for the SQL Script Engine and Tcl Script Engine Use, 12

Using Tcl Scripts and SQL Expressions in Data Replication

The Data Replication InitialSync and Applier components can process Tcl scripts and SQL expressions that you create to customize load processing.

To use Tcl scripts, you must be familiar with the Tcl scripting language. To use SQL expressions, you must be familiar with the SQL of the target database.

You can create Tcl scripts or SQL expressions for the following data replication purposes:

- Customize or convert target column datatypes.
- Perform advanced filtering of source data to apply to the target system.
- Perform mathematical operations on target data.
- Generate customized key values.
- Split a source column into multiple target columns.
- Perform aggregation functions on target data.
- Perform advanced logging during a data replication job.
- Execute SQL statements on the target database during a data replication job.
- Use a variety of functions for advanced text processing.

Data Replication supports Tcl scripts and SQL expressions for all types of replication jobs, including continuous replication.

Restrictions:

- Data Replication does not support SQL expressions for targets that use Audit Apply mode, such as Apache Kafka, Cloudera, Hortonworks, and Flat File targets.
- The Tcl Script Engine and SQL Script Engine do not support LOB data. Do not define Tcl scripts or SQL expressions that process LOB data.

Adding Tcl and SQL Expressions

You can add Tcl and SQL expressions to mapped source tables to customize InitialSync and Applier processing.

In the Data Replication Console, define a Tcl or SQL expression for a mapped source table and assign this expression to a virtual column. Then map the virtual column to a column in the target table. You can also copy virtual columns to other tables in the same configuration to reuse Tcl and SQL expressions.

Restrictions:

- Data Replication does not support SQL expressions for targets that use Audit Apply mode, such as Apache Kafka, Cloudera, Hortonworks, and Flat File targets.
- The Tcl Script Engine and SQL Script Engine do not support LOB data. Do not define Tcl scripts or SQL expressions that process LOB data.

Adding Tcl Expressions

After you write a Tcl script based on the information in the *Informatica Data Replication Scripting Guide*, enter the script for a source table in the Data Replication Console.

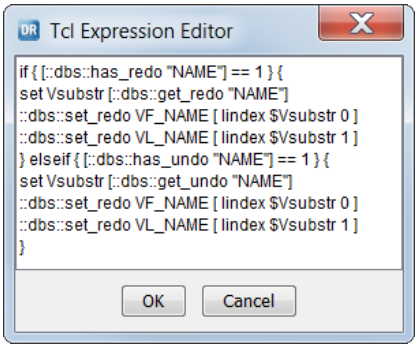
Before you begin, make sure that the Console is in Edit mode.

1. On the **Map Tables** tab, select the mapped source table row for which to define a Tcl expression.
2. Click the **Add or Show Table Expression** icon button above the source tables filter field, or right-click the table row and click **Add or Show Table Expression**.

   The **Add/Show expressions** dialog box appears.
3. Click **New**.
4. Enter an expression name and click **OK**.

   An expression name can be no longer than 50 characters and can include only the digits 0-9, Latin letters A-Z and a-z, and the underscore (_) character.
5. In the **Language** field, select **TCL**.
6. Double-click the **Expression** field or click **Edit**.

   The **Tcl Expression Editor** dialog box appears.



7. Enter a Tcl expression for processing the replicated data and click **OK**.

   The Tcl expression appears in the **Add/Show expressions** dialog box.

Tips:

- To edit an expression, double-click in the **Expression** field, or select the expression row and click **Edit**.
- To rename an expression, double-click in the **Name** field. Then edit the expression name and press **Enter**.
- To delete an expression that is not assigned to any virtual column, select the expression row and click **Delete**.

8. When done, click **Close**.

Next, you can assign the Tcl expression to a virtual column.

Adding SQL Expressions

In the Data Replication Console, you can use data access operators and SQL comparison and logical operators to quickly define an SQL expression.

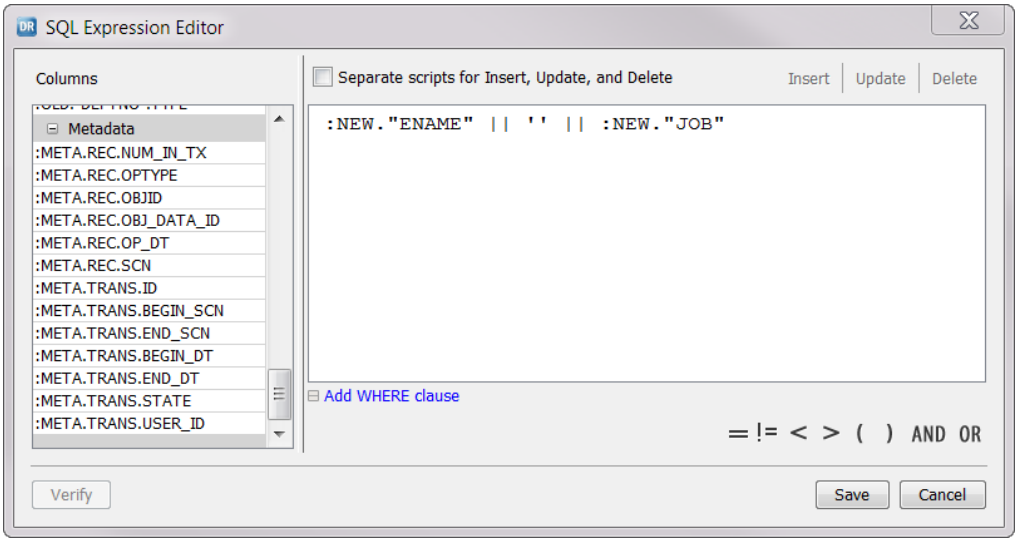
Before you begin, make sure that the Console is in Edit mode.

1. On the **Map Tables** tab, select the mapped source table row for which to define an SQL expression.
2. Click the **Add/Show Table Expression** icon button above the source tables filter field, or right-click the table row and click **Add/Show Table Expression**.

   The **Add/Show expressions** dialog box appears.
3. Click **New**.
4. Enter an expression name and click **OK**.

   An expression name can be no longer than 50 characters and can include only the digits 0-9, Latin letters A-Z and a-z, and the underscore (_) character.
5. In the **Language** field, select **SQL**.
6. Double-click the **Expression** field, or click the **Edit** button.

   The **SQL Expression Editor** dialog box appears.



7. To define separate SQL expressions for Insert, Update, and Delete operations, select the **Separate scripts for Insert, Update, and Delete** check box.

   To define a single SQL expression that affects all of these SQL operation types, do not select this check box.

8. Enter an SQL expression in the text box.

   If you selected the **Separate scripts for Insert, Update, and Delete** check box, use the **Insert**, **Update**, and **Delete** buttons to switch from one SQL operation type to another.

   To quickly create an SQL expression, you can use the data access operators in the **Columns** list box and the SQL operators in the bottom right corner. The Columns section of the **Columns** list contains columns in the mapped source table but does not contain virtual columns. Virtual columns cannot be included in an SQL expression. To insert a data access operator, click the operator row. To insert an SQL operator, click the button for the operator.

   Note: Verify that all single and double quotation marks in the SQL expression have the appropriate closing quotation marks. If you save a replication configuration that includes an SQL expression in which one or more closing quotation marks are missing, the Data Replication Console will incorrectly display this SQL expression when you open the configuration later.

9. For Updates and Deletes, if you want to map the virtual column to a primary key column on the target, click **Add WHERE clause**. Then under **WHERE clause**, enter a WHERE clause.

10. Click **Save**.

    The SQL expression appears in the **Add/Show Expression** dialog box.

    Tips:

    - To edit an expression, double-click in the **Expression** field, or select the expression row and click **Edit**.
    - To rename an expression, double-click in the **Name** field. Then edit the expression name and press **Enter**.
    - To delete an expression that is not assigned to any virtual column, select the expression and click **Delete**.

11. When you are finished, click **Close**.

After you assign the defined expression to a virtual column and map the virtual column to a target column, you can open the **SQL Expression Editor** dialog box again and click **Verify** to validate the SQL expression.

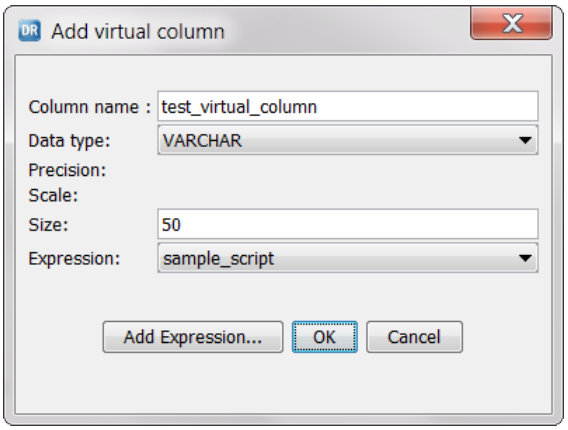
Adding a Virtual Column with an SQL or Tcl Expression

After you define an SQL or Tcl expression for a source table, you must assign it to a virtual column. Then map the virtual source column to a column in the corresponding target table.

Important: Data Replication validates that the virtual column and target column have compatible datatypes. If the datatypes are not compatible, the Data Replication Console displays an error message that prompts you to map the virtual column to a target column that has a compatible datatype. For more information about compatible datatypes, see the *Informatica Data Replication Datatype Mapping Reference*.

1. On the **Map Columns** tab, in the **Source Table** list, select the source table for which you previously defined a Tcl or SQL expression.
2. Click the **Add Virtual Column** icon button above the source columns box, or right-click any source column row and click **Add Virtual Column**.

The **Add virtual column** dialog box appears.



3. In the **Column name** field, enter a unique virtual column name.

   A virtual column name can be no longer than 50 characters and can include only the digits 0-9, Latin letters A-Z and a-z, and the underscore (_) character.

   Note: Do not create a virtual column that has the same name as another virtual column in the same table.

4. In the **Data type** list, select the virtual column datatype.

   The following table describes the virtual column datatypes:

Datatype	Description
BIGINT	Big integers. Storage size: 8 bytes Range of values: -2^63 (-9223372036854775808) through 2^63-1 (9223372036854775807)
DATE	Combined date and time value. Date format for Tcl scripts: yyyy-mm-dd hh:mm:ss Date format for SQL expressions: target database format
DECIMAL(*p*,*s*)	Decimal numbers with the declared precision and scale. Scale must be less than or equal to precision. Valid precision values: 1 through 38 Valid scale values: 0 through 38
DOUBLE	Double-precision floating-point numbers. Storage size: 8 bytes Range of values: -7.2E+75 through 7.2E+75
FLOAT	Single-precision floating-point numbers. Storage size: 4 bytes Range of values: -3.40E + 38 through -1.18E - 38, 0 and 1.18E - 38 through 3.40E + 38
INTEGER	Large integers. Storage size: 4 bytes Range of values: -2^31 (-2147483648) through 2^31 - 1 (2147483647)

Datatype	Description
NVARCHAR(*n*)	Variable-length Unicode data. Valid length values: 1 through 4000 bytes
TIMESTAMP(*f*)	Date and time value that includes the year, month, day, hour, minutes, and seconds. Timestamp format for Tcl scripts: yyyy-mm-dd hh:mm:ss ff Timestamp format for SQL expressions: target database format The *f* value is the number of digits in the fractional part of seconds. Valid fractional seconds precision values: 0 through 9
VARCHAR(*n*)	Variable-length non-Unicode data. Valid length values: 1 through 4000 bytes

5. In the **Expression** list, select the Tcl or SQL expression that you previously defined.
6. Click **OK** to save the column-expression association.

   Tip: To view virtual columns that use a particular expression, in the **Add/Show expressions** dialog box, right-click the expression row and click **List virtual columns**.
7. On the **Map Columns** view, map the virtual source column with the Tcl or SQL expression to the corresponding target column. These columns must have compatible datatypes.

   For more information about recommended datatype mappings for virtual source columns, see the *Informatica Data Replication Datatype Mapping Reference*.

   When you run the InitialSync or the Applier component later, the component evaluates the expression for the virtual column and writes the result to the corresponding target column.

Defining a Virtual Index for the SQL Script Engine and Tcl Script Engine Use

For the SQL Script Engine and Tcl Script Engine to replicate Update statements correctly, you must define a virtual index with the **Supplemental logging** option. Include all of the source columns that you access in the Tcl script or SQL expression in this virtual index.

For Oracle sources, the Data Replication Console enables supplemental logging for the columns that are included in the virtual index with the **Supplemental logging** option and for the primary key columns. When the Oracle Extractor processes an Update, the Extractor writes all of the extracted before- and after-image values to the intermediate files.

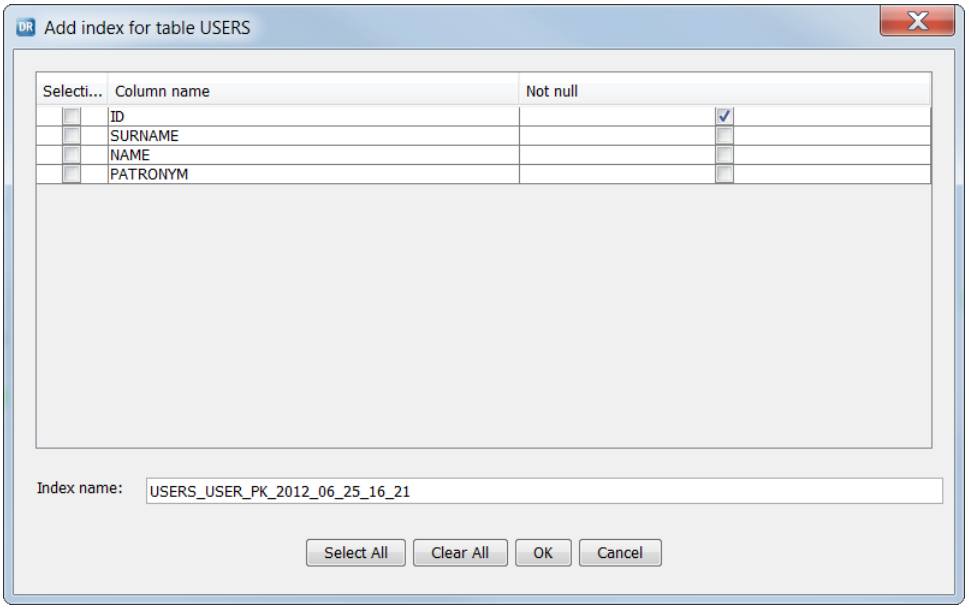
For the sources that enable additional logging on the table level, such as DB2 and Microsoft SQL Server sources, the Extractor by default writes the before- and after-image values only for the primary key columns and updated columns to the intermediate files. If you define a virtual index with the **Supplemental logging** option, the Extractor also writes the before- and after-image values for the columns that are included in the index to the intermediate files.

When the Applier processes an Update, it evaluates the virtual column expression even if none of the columns that are used in the expression were updated. To correctly process the virtual column expression, the Applier requires the before- and after-image values for all of the columns that are used in the expression to be in the intermediate files.

Note: Because the Oracle Extractor writes all of the extracted values to the intermediate files, you do not need to define a virtual index with the **Supplemental logging** option if you manually enabled supplemental logging.

1. On the **Map Tables** tab, select a mapped source table for which to add a virtual index.
2. Click **Add**.

   The **Add index for table <table name>** dialog box appears.



3. Select the columns to use for the virtual index, or click **Select All** to select all of the columns.

   Note: You cannot edit the **Not null** setting.
4. In the **Index name** field, enter a name for the index or accept the default name.

   Index names can contain only the digits 0-9, Latin letters A-Z and a-z, and the underscore (_) character.
5. Click **OK**.

   The **Indexes** dialog box lists the new index.
6. Clear the **Unique** and **Primary key** options for the added index.
7. Verify that the **Supplemental logging** and **User Defined** options are selected.
8. Click **OK**.

CHAPTER 2

Tcl Scripting

This chapter includes the following topics:

- Using Tcl Scripts, 14
- ::dbs:: Tcl Namespace Reference, 18
- Examples of Tcl Scripts, 26

Using Tcl Scripts

Data Replication can use Tcl scripts that you assign to virtual columns defined for a source database to calculate values in the target database during InitialSync and Apply processing.

Tcl Script Execution

After you define Tcl scripts for a table in the Data Replication Console, the Tcl scripts run as part of the scheduled replication job during Apply processing or as part of the InitialSync job.

Note: InitialSync does not support Apache Kafka or flat-file target types.

Tcl creates a single interpreter for each Apply task or InitialSync thread. For example, if you have four Apply task threads, Tcl uses four independent interpreters.

If you configured the replication job to use multiple subtask threads for an Apply task, Tcl creates a single interpreter for each subtask thread. You enable subtask threads by setting the apply.subtasks_enabled parameter to 1 on the **Runtime Settings** tab > **Advanced Settings** view.

For Teradata, Netezza, PostgreSQL, and Greenplum targets, Data Replication can use either a native load utility or the appropriate ODBC driver for initial materialization of target tables. For Microsoft SQL Server and DB2 targets, InitialSync cannot use the native load utility with Tcl expressions that calculate column values for the target tables. For these targets, set the initial.enforce_odbc_load parameter to 1.

Accessing Input Data

The Tcl Script Engine provides two methods of accessing input data. You can use either one.

Method 1: Process all input columns as a single Tcl dictionary object. The input columns are organized as pairs, where the key is a name of the column. This organization supports group access and operations.

To extract and populate the variables in a script scope, use the following expression:

```
::dbs::get_undo [ dict get $::dbs::repo "< column_name >" ]
```

The following example shows a typical command:

```
set curr_call_date [ ::dbs::get_undo[ dict get $::dbs::repo "CALL_DATE" ]]
```

Alternatively, use a foreach technique to iterate through all '*input_column*' objects and execute the loop body, for example:

```
dict for {name i_c} $::dbs::repo{
puts " Column 'undo' - [::dbs::get_undo $i_c ] "
puts " Column 'redo' - [::dbs::get_redo $i_c ] "
}
```

Method 2: Access the input data by using the ::dbs::get_undo command without a ::dbs::repo variable. Instead, use a column name string as the argument, for example.

```
::dbs::get_undo CALL_DATE
```

The second argument of ::dbs::get_undo command does not have to be the string. You can use the command with another language element such as an expression, for example:

```
::dbs::get_undo [some_proc some_arg]
```

In this case, the Tcl Script Engine automatically extracts the string representation of the value that is returned from [some_proc some_arg] call.

Related Topics:

- "Change Metadata and Applier State Commands and the ::dbs::repo Variable" on page 23

Setting the Virtual Column Value

Use the ::dbs::set_redo <*column_name*> <*object*> and ::dbs::set_undo <*column name*> <*object*> commands to set the redo and undo values of a virtual column that is named <*column_name*>.

In Tcl terms, these commands create special variables named <*column_name*> of the type '*output_column_type*' in the expression scope. The variables have some usage limitations. Copy and delete operations on type conversions are not allowed.

The following example shows a typical command:

```
::dbs::set_redo Virtcol1 $var
```

Special cases: Variables and objects of '*output_column_type*' are Tcl variables that have special limitations on usage. To be processed properly, they should be created by special commands. Because of performance considerations, no symbolic analysis and name lookup is performed before actual script processing. The same situation occurs with the call.

The following command that defines "Virt1" as a virtual column is valid:

```
set Virt1 $a
```

The Virt1 variable is created with the type and value that is stored in the "a" variable. However, the variable is not mapped to the target. When a variable is associated with a type other than '*output_column_type*', all of the standard operations are permitted.

Tcl Script Engine Processing Stages

The Tcl Script Engine runs Tcl script files or expressions during the appropriate Applier or InitialSync initialization, data processing, and finalization stages.

1. When an Applier or InitialSync task initializes, the engine runs the ./support/tcl/dbs_main_start.tcl script.

2. When an Applier or InitialSync thread initializes, the engine runs the ./support/tcl/dbs_worker_start.tcl script.
   The ./support/tcl/dbs_worker_start.tcl file includes all initialization code for each thread. The engine also checks for specific library and package requirements or local variables that are associated with local thread context.
3. When the Applier starts an apply cycle, the engine runs the ./support/tcl/dbs_cycle_start.tcl script.

   Important: The engine does not run Tcl expressions that are defined in the dbs_cycle_start.tcl script for InitialSync tasks.
4. During the Applier or InitialSync main data processing stage, the engine runs the Tcl expressions for virtual columns that you added to source and target tables.
   You can add a virtual column with an expression from the **Map Tables** tab of the Data Replication Console. Right-click a source table and click **Add Virtual source column**. In the expression, include all of the data manipulation code.
5. When the Applier ends an apply cycle, the engine runs the ./support/tcl/dbs_cycle_end.tcl script.

   Important: The engine does not run Tcl expressions that are defined in the dbs_cycle_end.tcl script for InitialSync tasks.
6. When an Applier or InitialSync thread ends, the engine runs the ./support/tcl/dbs_worker_end.tcl script. This finalization script can produce reports or perform special functions such as running SQL statements or sending email notifications.
7. When the Applier or InitialSync task ends, the engine runs the ./support/tcl/dbs_main_end.tcl script.

Important: For apply processing, the Tcl Script Engine can run the dbs_main_start.tcl, dbs_main_end.tcl, dbs_cycle_start.tcl, and dbs_cycle_end.tcl scripts only if you set the apply.script_engine_enabled parameter to 1 on the **Runtime Settings** tab > **Advanced Settings** view in the Data Replication Console.

Stages 1 , 3, 5, and 7 run within a main thread. The input and output commands that are related to change record metadata are not available,
including ::dbs::get_redo, ::dbs::get_undo, ::dbs::repo, ::dbs::set_undo, ::dbs::set_redo, ::dbs::has_undo, ::dbs::has_redo, and ::dbs::get_type.

Related Topics:

- "Global Context" on page 17

Scripting Considerations

The most efficient way to organize scripting is to place input and output assignments in the expression body and place the processing procedure in a separate script file.

For example, the following statement specifies the script file:

```
#to load extra .tcl file we should use 'source' command with full path
 source./support/tcl/dbs_worker.tcl
```

The Tcl Script Engine is not aware of the Data Replication apply mode that is used. The engine works with a pair of undo and redo values that are associated with processing a column. To get or set these values, use the following commands:

- ::dbs::get_undo
- ::dbs::get_redo
- ::dbs::set_undo
- ::dbs::set_redo

You might be able to place the entire script in the expression body, depending on the type of the processing. The following examples show scripts that can be included in the expression body:

- If the replication captures only inserts, the script does not need to process undo records as well as redo records. The following example script reflects this situation:

```
;# entire script placed in expression body
source foo.tcl
::dbs::set_redo V1 [ foo [::dbs::get_redo COLUMN1] ]
```

- If the replication also captures updates and deletes, the script must process undo and redo records. The following example script reflects this situation:

```
;# entire script placed in expression body
source foo.tcl
::dbs::set_undo V1 [ foo [::dbs::get_undo COLUMN1] ]
::dbs::set_redo V1 [ foo [::dbs::get_redo COLUMN1] ]
```

Global Context

For applications with heavy workloads, you might be able to significantly improve performance by running multiple interpreters on separate Applier threads.

In this case, the interpreters can interact with each other by sharing special variables in threaded context storage. This storage can store virtually any database or native type, except Tcl container objects.

To create, set, or get this special variable, use the ::dbs::set_glob and ::dbs::get_glob commands. To clear this variable, use the ::dbs::unset_glob command. The following example gets and sets the variable:

```
if { [ ::dbs::get_glob count ] == 0 } {
::dbs::set_glob count 1
}

set Var1 [ ::dbs::get_glob count ]
```

Use the ::dbs::chk_glob command to ensure that the variable exists in global context. For example:

```
if { [ ::dbs::chk_glob some_string ] == 0 } {
::dbs::set_glob some_string "string has been placed to global context storage."
}
```

You can use the ::dbs::get_glob, ::dbs::chk_glob, ::dbs::set_glob, and ::dbs::unset_glob commands to work with containers in threaded context storage. Use the ^ symbol to indicate maps.

Use the ::dbs::set_glob command to create maps and nested maps and to add objects to maps to threaded context storage.

- The following example uses the ::dbs::set_glob command to create a map object named hash1:

```
::dbs::set_glob ^hash1
```

- The following example uses the ::dbs::set_glob command to add the pair *some_key*<=>*some_data* to the previously created map hash1:

```
::dbs::set_glob ^hash1 $some_key $some_data
```

  The map hash1 must exist when you execute this command. Otherwise, the Tcl Script Engine reports an error.

- The following example uses the ::dbs::set_glob command to create the nested map hash2:

```
::dbs::set_glob ^hash1 ^hash2
```

  The map hash2 is nested in the map hash1. The map hash1 must exist when you execute this command.

To extract values from maps that are stored in a global context, use the ::dbs::get_glob command with two arguments. The first argument is the map name, and the second argument is the key name.

- The following example extracts a value that is stored in the previously created map hash1, which is referenced by the key *some_key* in a global context:

  ```
  ::dbs::get_glob ^hash1 $some_key
  ```

  The command returns an object that you added previously and that is stored in the *some_data* variable. If you specify a map that does not exist, the Tcl Script Engine reports an error. If you specify a key that does not exist for an object, the Tcl Script Engine reports an error.

To remove an object from the map in threaded context storage, use the ::dbs::unset_glob command.

- The following example uses the ::dbs::unset_glob command to remove the value that is referenced by *some_key* key from the map named hash1, which is stored in a global context:

  ```
  ::dbs::unset_glob ^hash1 $some_key
  ```

  The specified map must exist in the global context. Also, the object that is associated with the specifed key must exist in the global context.

Script Execution Control

During apply processing you can use the ::dbs::skip command to skip the current change record or use the ::dbs::abort command to stop the Applier process immediately.

The following example extracts the value of the "COL1" column, divides the value by 2, and if the remainder is a number other than 0, skips the current iteration:

```
set var_chk [ ::dbs::get_undo COL1 ]
if { [ expr $var_chk % 2 ] != 0 } {
::dbs::skip
}
```

Important: The Tcl Script Engine skips the ::dbs::skip and ::dbs::abort commands during the InitialSync synchronization of source and target tables.

::dbs:: Tcl Namespace Reference

The ::dbs:: Tcl namespace extends basic Tcl functionality by defining a collection of commands that you can use in Data Replication apply processing.

Data Replication also provides the ::dbs:: extension of Tcl datatypes. The ::dbs:: extension can process datatypes such as timestamp, datetime, and timestamp with time zone.

You can use ::dbs:: commands to customize some target column datatypes, control apply processing, assess the processing state, and perform mathematical and compare operations on data objects.

I/O Utility Commands

::dbs::set_undo <*virtual_column_name*> <*script_scope_variable_name*>

Allocates memory and puts the < *script_scope_variable_name* > value in the undo field of the virtual column object. No value is returned.

::dbs::set_redo <*virtual_column_name*> <*script_scope_variable_name*>

Allocates memory and puts the < *script_scope_variable_name* > value in the redo field of the virtual column object. No value is returned.

::dbs::get_undo <*input_column_name*>

Extracts the undo value of the specified input column and returns an object with the value in the same datatype as the input column.

::dbs::get_redo <*input_column_name*>

Extracts the redo value of the specified input column and returns an object with the value in the same datatype as the input column.

::dbs::get_undobytes <*input_column_name*>

Extracts the undo value of the specified input column and returns a Tcl byte array object with the exact memory representation of the undo value.

::dbs::get_redobytes <*input_column_name*>

Extracts the redo value of the specified input column and returns a Tcl byte array object with the exact memory representation of the redo value.

::dbs::has_undo <*input_column_name*>

Returns true if the specified input column contains undo data. Otherwise, returns false.

::dbs::has_redo <*input_column_name*>

Returns true if the specified input column contains redo data. Otherwise, returns false.

::dbs::record::undo_is_null <*input_column_name*>

For Applier processing, returns true if the undo data for the specified input column is null. Otherwise, returns false. For InitialSync processing, returns a null value.

::dbs::record::redo_is_null <*input_column_name*>

For Applier processing, returns true if the redo data for the specified input column is null. Otherwise, returns false. For InitialSync processing, returns a null value.

::dbs::get_type <*input_column_name*>

Returns a string object that contains the datatype of the specified input column.

Datatype Object Commands

::dbs::decimal <*value*>

Allocates and returns an object of the decimal datatype. To set Tcl variables, use the following syntax:

```
set Var [::dbs::decimal 0.001]
```

::dbs::datetime <value>

Allocates and returns an object of the date datatype that stores the specified value. The value must be specified in the following format:

```
YYYY-MM-DD hh:mm:ss
```

To set Tcl variables, use the following syntax:

```
set Var [::dbs::datetime 2011-05-31 12:00:00 ]
```

::dbs::timestamp <value>

Allocates and returns an object of the timestamp datatype that stores the specified value. The value must be specified in the following format:

```
YYYY-MM-DD hh:mm:ss nanoseconds
```

To set Tcl variables, use the following syntax:

```
set Var [::dbs::timestamp 2011-05-31 12:00:00 43423 ]
```

::dbs::timestamp_tz <*timestamp_object*> <*integer_representation_of_time_zone*>

Allocates and returns an object of the timestamp with time zone datatype. To set Tcl variables, use the following syntax:

```
set Var [::dbs::timestamp_tz $ts $tz ]
```

Where *ts* and *tz* are timestamp object and integer replesentation of time zone. For example, you can use the following command:

```
set Var [::dbs::timestamp_tz [::dbs::timestamp 2011-05-31 12:00:00 1000 ] 5 ]
```

::dbs::extract_tz <*timestamp_with_timezone_object*>

Returns an integer value that represents the time zone.

::dbs::system::current_timestamp

Returns the system timestamp value.

::dbs::system::current_datetime

Returns the system datetime value.

::dbs::add_y <*object*> <*integer_value*>

Modifies an object that has the datetime or timestamp datatype by adding a year value as an integer.

::dbs::add_mo <*object*> <*integer_value*>

Modifies an object that has the datetime or timestamp datatype by adding a month value as an integer.

::dbs::add_d <*object*> <*integer_value*>

Modifies an object that has the datetime or timestamp datatype by adding a days value as an integer.

::dbs::add_h <*object*> <*integer_value*>

Modifies an object that has the datetime or timestamp datatype by adding an hours value as an integer.

::dbs::add_mi <*object*> <*integer_value*>

Modifies an object that has the datetime or timestamp datatype by adding a minutes value as an integer.

::dbs::add_s <*object*> <*integer_value*>

Modifies an object that has the datetime or timestamp datatype by adding a seconds value as an integer.

::dbs::sub_y <*object*> <*integer_value*>

Modifies an object that has the datetime or timestamp datatype by subtracting the year integer value.

::dbs::sub_mo <*object*> <*integer_value*>

Modifies an object that has the datetime or timestamp datatype by subtracting the month integer value.

::dbs::sub_d <*object*> <*integer_value*>

Modifies an object that has the datetime or timestamp datatype by subtracting the days integer value.

::dbs::sub_h <*object*> <*integer_value*>

Modifies an object that has the datetime or timestamp datatype by subtracting the hours integer value.

::dbs::sub_mi <*object*> <*integer_value*>

Modifies an object that has the datetime or timestamp datatype by subtracting the minutes integer value.

::dbs::sub_s <*object*> <*integer_value*>

Modifies an object that has the datetime or timestamp datatype by subtracting a seconds integer value.

Usage notes:

- Database types such as native integers, floating point numbers, and strings such as VARCHAR are converted to Tcl native datatypes so that they can be assigned to a Tcl variable and processed without the ::dbs:: extension of datatypes. For example:

  ```
  set i [::dbs::get_redo IN_COL1]
  ```

  In this example, if the IN_COL1 column has the native datatype of integer, the Tcl variable "i" is created as a native integer and all standard Tcl operations are applicable.
- The following example shows a typical usage of the ::dbs::add_h command:

  ```
  ::dbs::add_h $some_data 1
  ```

Tcl Script Engine Interaction Commands

::dbs::is_applier

Returns 1 for Applier processing. Returns 0 for InitialSync processing.

::dbs::is_initialsync

Returns 1 for InitialSync processing. Returns 0 for Applier processing.

::dbs::set_glob <*variable_name* > <*object*>

Stores the object in the specified variable of the global context.

::dbs::set_glob ^<*map_name*>

Creates a map in the global context. The ^ symbol precedes the map name.

::dbs::set_glob ^<*map_name1*>… ^<*map_nameN*>

Creates a map that is recurrsively nested in other maps in the global context. The ^ symbol precedes each map name.

::dbs::set_glob ^<*map_name1*>… ^<*map_nameN*> <*key_name*> <object>

Puts the key name and object into a map, which can be recursively nested into other maps. You must have previously created all of the maps that are specified in the command.

::dbs::get_glob <*variable_name*>

Returns the object that is stored in the specified variable of the global context.

::dbs::get_glob ^<*map_name1*>… ^<*map_nameN*> <*key_name*>

Returns the object with the specified key from the map in the global context. The map can be recursively nested into other maps. An error occurs if the map does not exist.

::dbs::chk_glob <*variable_name*>

If the specified variable exists, returns the integer value of the type ID for the object that is stored in the variable. Otherwise, returns 0.

::dbs::chk_glob ^<*map_name1*>… ^<*map_nameN*> <*key_name*>

Returns the integer value of the type ID with the specified key from the map in the global context. The map can be recursively nested in other maps. An error occurs if the map does not exist.

::dbs::unset_glob <*variable_name*>

Unsets the object stored in the specifed variable.

::dbs::unset_glob ^<*map_name1*>… ^<*map_nameN*> <*key_name*>

Unsets the object associated with the specified key in the map in threaded context storage.

::dbs::log <*option*> <*string_object*>

Sends a message string to the Data Replication Applier for logging. Valid options are:

- -i. Sends a message that has the LM_INFO severity level.
- -d. Sends a message that has the LM_DEBUG severity level.
- -w. Sends a message that has the LM_WARNING severity level.
- -n. Sends a message that has the LM_NOTICE severity level.
- -e. Sends a message that has the LM_ERROR severity level.
- -c. Sends a message that has the LM_CRITICAL severity level.
- -a. Sends a message that has the LM_ALERT severity level.

::dbs::lock_glob

Enters the critical section.

::dbs::unlock_glob

Exits the critical section

::dbs::abort

Stops the Applier process.

::dbs::skip

Skips the current change record.

Data Manipulation Commands

::dbs::datetime_comp <*datetime_or _timestamp_object1*> <*comparison_operator*> <*datetime_or_timestamp_object 2*>

Performs a comparison of two datetime or timestamp objects. Returns a Boolean true or false object. The *comparison operator* can be one of the following operators:

- >
- <
- <=
- >=
- ==

The following example command compares two date objects, and if they are not equal, writes the string "$date1 and $date2 are not equal!" to the console:

```
if {[ ::dbs::datetime_comp $date1!= $date2 ]}{
 puts "$date1 and $date2 are not equal!"
}
```

::dbs::datetime_extract <*option*> <*datetime_or_timestamp_object*>

Extracts date or time elements from a datetime or timestamp object as integers. Valid *option* values are:

- -y. Extracts the year.
- -mo. Extracts the month.
- -d. Extracts the day.

- -h. Extracts hours.
- -m. Extracts minutes.
- -s. Extracts seconds.
- -ms. Extracts milliseconds.
- -ns. Extracts nanoseconds.

Change Metadata and Applier State Commands and the ::dbs::repo Variable

The Tcl Script Engine supports commands for retrieving information about change records, transactions, and threads for Applier processing. It also supports a global variable for Applier and InitialSync group operations with lists and a dictionary of input column objects.

For example, use the commands to determine which transactions started or were committed prior to a specific date and time.

Commands

::dbs::record::op_type

Returns the DML operation type that is associated with the record. For Applier processing, the operation type is one of the following values:

- **I** for Inserts.
- **U** for Updates.
- **D** for Deletes.

For InitialSync processing, this value is **O** for Inserts.

::dbs::change_optype <*option*>

Changes the DML operation type that is associated with the record. Valid *option* values are:

- **I**. Change the record operation type to Insert.
- **D**. Change the record operation type to Delete.
- **U**. Change the record operation type to Update.

Note: In Merge Apply mode, if you change the operation type from Delete to Update, the Applier applies null values to all target columns for the record.

::dbs::record::obj_id

For Applier processing, returns the integer value of the current table object ID associated with the change record.

For InitialSync processing, returns a null value.

::dbs::record::op_time

For Applier processing, returns the datetime object that contains the date and time when the operation that is associated with a record occurred.

For InitialSync processing, returns a null value.

::dbs::system -<*option*>

This command is deprecated.

::dbs::transaction::start_dt

For Applier processing, returns the datetime object that contains the date and time when the transaction started.

For InitialSync processing, returns a null value.

::dbs::transaction::end_dt

For Applier processing, returns the datetime object that contains the date and time when the transaction committed.

For InitialSync processing, returns a null value.

::dbs::transaction::state

Returns the transaction state. For Applier processing, the transaction state can be one of the following values: **R** for rollback, **A** for activated, or **C** for committed.

For InitialSync processing, returns a null value.

::dbs::transaction::beginscn

For Applier processing, returns the native string representation of the start SCN for the transaction.

For InitialSync processing, returns a null value.

::dbs::transaction::endscn

For Applier processing, returns the native string representation of the commit SCN for the transaction.

For InitialSync processing, returns a null value.

::dbs::transaction::transaction_id

For Applier processing, returns the native representation of the transaction ID.

For InitialSync processing, returns a null value.

::dbs::transaction::user_id

For Applier processing, returns the user ID of the database user who performed a DML operation.

For Applier processing of change data from DB2 sources, returns a null value.

For InitialSync processing, returns a null value.

::dbs::transaction::user_name

For Applier processing, returns the name of the database user who performed a DML operation.

Note: If the transaction logs do not contain the user name, returns a null value.

For InitialSync processing, returns a null value.

::dbs::interp_id

For Applier processing, returns the integer value of the interpreter ID. For InitialSync processing, returns a null value.

::dbs::thread_id

For Applier processing, returns the system thread ID.

For InitialSync processing, returns a null value.

Global Variable

::dbs::repo

A global variable that stores a standard Tcl dictionary object, which contains the hash pair "<*input_column_name*>" <=> *input_column_index*.

This variable is available only in the main processing stage of the Applier or InitialSync task. Use the variable to perform group operations with lists and a dictionary of input column objects such as foreach.

Related Topics:

- "Accessing Input Data" on page 14

SQL Execution Commands

Data Replication provides commands in the ::dbs:: namespace that you can use to execute SQL commands for a target during Applier processing.

::dbs::create_stream -d <*SQL_statement*>

For Applier processing, creates the SQL stream object based on a SQL statement string that is written using the Oracle, ODBC and DB2-CLI Template Library, Version 4.0 (OTL 4.0) syntax. The command returns the integer ID of the SQL statement stream object for further use.

::dbs::delete_stream <*stream_ID*>

For Applier processing, deletes the SQL statement stream object that has the specified ID.

::dbs::bind_in <*stream ID*> <*string_object*>

For Applier processing, binds the string to the SQL statement stream placeholder. Call this function for each stream placeholder. For example, if SQL statement stream contains two placeholders, execute this command twice.

::dbs::bind_out <*stream_ID*>

For the SELECT statement stream, returns the current string object from the stream during Applier processing. The stream uses an iterator to iterate through the result string objects.

::dbs::flush <*option*>

For Applier processing, flushes SQL statement streams. You can use the following options:

- -all. Flushes all opened streams of the Applier, except for streams opened by the user.
- -s. Flushes all streams opened in the current thread, except for streams opened by the user.

::dbs::flush <*stream_ID*>

For Applier processing, flushes the SQL statement stream that has the specified ID if it was opened by the current user.

::dbs::check_eof <*stream_ID*>

For Applier processing, returns true if the end of the stream that has the specified ID has been reached. Otherwise, returns false.

::dbs::fetch_result <*stream_ID*>

For Applier processing, returns a Tcl list object, which represents the SQL statement result. Typically, the command returns a list of records that were selected by SELECT statement.

Warning: When InitialSync encounters any of these commands, it ends with an error. To exclude these commands from InitialSync processing, add a condition that checks the current operation type to determine

if it is an InitialSync operation. For example, the following condition causes the Tcl script to flush SQL statement streams only when the operation type does not equal O (InitialSync Insert):

```
if {[::dbs::record::optype] != 'O'} {::dbs::flush -s}
```

Examples of Tcl Scripts

Review these examples to learn how to code and use Tcl scripts. The examples demonstrate how to perform some basic operations using the commands in the ::dbs:: Tcl namespace.

Getting Started with Some Simple Scripts

To get started, review some simple Tcl script examples.

- The following example splits the value stored in the NAME source column into a pair of last name and first name values:

  ```
  if { [::dbs::has_redo NAME] == 1 } {
  set Vsubstr [::dbs::get_redo NAME]
  ::dbs::set_redo VF_NAME [ lindex $Vsubstr 0 ]
  ::dbs::set_redo VL_NAME [ lindex $Vsubstr 1 ]
  } elseif { [::dbs::has_undo NAME] == 1 } {
  set Vsubstr [::dbs::get_undo NAME]
  ::dbs::set_redo VF_NAME [ lindex $Vsubstr 0 ]
  ::dbs::set_redo VL_NAME [ lindex $Vsubstr 1 ]
  }
  ```

  In this example:

  - NAME is the source table column.
  - VF_NAME and VL_NAME are virtual source columns.
  - lindex is a Tcl function that skips the spaces between the words. This function is defined in a file other than the script file.

- The following example concatenates the first name in the F_Name source column and the last name in the L_Name source column into a single FULL_NAME target column with a space between the names:

  ```
  if { [::dbs::has_redo F_NAME] == 1 && [::dbs::has_redo L_NAME] == 1} {
  set Vsubstr "[::dbs::get_redo F_NAME] [::dbs::get_redo L_NAME]"
  ::dbs::set_redo FULL_NAME $Vsubstr
  } elseif { [::dbs::has_undo F_NAME] == 1 && [::dbs::has_redo L_NAME] == 1} {
  set Vsubstr "[::dbs::get_undo F_NAME] [::dbs::get_redo L_NAME]"
  ::dbs::set_redo FULL_NAME $Vsubstr
  } elseif { [::dbs::has_redo F_NAME] == 1 && [::dbs::has_undo L_NAME] == 1} {
  set Vsubstr "[::dbs::get_redo F_NAME] [::dbs::get_undo L_NAME]"
  ::dbs::set_redo FULL_NAME $Vsubstr
  } elseif { [::dbs::has_undo F_NAME] == 1 && [::dbs::has_undo L_NAME] == 1} {
  set Vsubstr "[::dbs::get_undo F_NAME] [::dbs::get_undo L_NAME]"
  ::dbs::set_redo FULL_NAME $Vsubstr
  }
  ```

  In this example:

  - F_NAME and L_NAME are source table columns.
  - FULL_NAME is a source virtual column.

- The following example demonstrates string extraction, concatenation, and positioning to create a full name with only the first character of the middle name followed by a period:

  ```
  if { [::dbs::has_redo SOME_CHAR] == 1 } {
  set Vsubstr [::dbs::get_redo SOME_CHAR]
  ```

```
set Vres "[ lindex $Vsubstr 0 ] [ string index [ lindex $Vsubstr 1 ] 0 ]. [lindex
$Vsubstr 2]"
::dbs::set_redo FULL_NAME $Vres
} elseif { [::dbs::has_undo SOME_CHAR] == 1 } {
set Vsubstr [::dbs::get_undo SOME_CHAR]
set Vres "[ lindex $Vsubstr 0 ] [ string index [ lindex $Vsubstr 1 ] 0 ]. [lindex
$Vsubstr 2]"
::dbs::set_redo FULL_NAME $Vres
}
```

In this example:

- SOME_CHAR is a source table column.
- FULL_NAME is a source virtual column.

- The following example does not replicate data if a source field has the value of "West" or "East":

```
if { [::dbs::get_redo SOME_CHAR] == "West" || [::dbs::get_redo SOME_CHAR] == "East" }
{
::dbs::skip
}
```

In this example, SOME_CHAR is a source table column.

Note: Execute this expression after all other expressions. You can attach the expression to the any virtual column. You do not need to map it.

- The following example adds the current system date and time for replication to the DATE virtual source column:

```
set Vtime [::dbs::datetime [clock format [clock seconds] -format "%Y-%m-%d"] [clock
format [clock seconds] -format "%H:%M:%S"]]
::dbs::set_redo DATE $Vtime
```

- The following example replaces the */home/users/account* path with the */home2/users2/account* path:

```
if { [::dbs::has_redo SOME_CHAR] == 1 } {
set Var3 "[string map {home/users home2/users2} [::dbs::get_redo SOME_CHAR]]"
::dbs::set_redo MAPS $Var3
} elseif { [::dbs::has_undo SOME_CHAR] == 1 } {
set Var3 "[string map {home/users home2/users2} [::dbs::get_undo SOME_CHAR]]"
::dbs::set_redo MAPS $Var3
}
```

In this example:

- SOME_CHAR is a source table column.
- MAPS is a virtual column.

- The following example evaluates if a column value is equal to one or many values:

```
set forskip "No"
if { [::dbs::has_redo SOME_NUMBER] == 1 } {
set vswitch [::dbs::get_redo SOME_NUMBER]
switch "$vswitch" {
"1" {::dbs::set_redo TEST [::dbs::decimal [::dbs::get_redo SOME_NUMBER]] }
"2" {::dbs::set_redo TEST [::dbs::decimal [::dbs::get_redo SOME_NUMBER]] }
"3" {::dbs::set_redo TEST [::dbs::decimal [::dbs::get_redo SOME_NUMBER]] }
default {set forskip "Yes" }
}
} elseif { [::dbs::has_undo SOME_NUMBER] == 1 } {
set vswitch [::dbs::get_undo SOME_NUMBER]
switch "$vswitch" {
"1" {::dbs::set_redo TEST [::dbs::decimal [::dbs::get_undo SOME_NUMBER]] }
"2" {::dbs::set_redo TEST [::dbs::decimal [::dbs::get_undo SOME_NUMBER]] }
"3" {::dbs::set_redo TEST [::dbs::decimal [::dbs::get_undo SOME_NUMBER]] }
default {set forskip "Yes" }
}
}
## this block plased after all expressions
if { $forskip == "Yes" } {
```

```
::dbs::skip
}
```

In this example:

- SOME_NUMBER is a source table column.
- TEST is a virtual column.

- The following example retrieves source system values for the Apply mode and SQL operation type (insert, update, or delete):

```
set Var2 "[::dbs::system -pm] [::dbs::system -ot]"
::dbs::set_redo SYS_INFO $Var2
```

In this example, SYS_INFO is a source virtual column.

- The following example dynamically executes SQL commands on the target system to retrieve the undo source from the current source row, use that value to read another row in the target table, and replicate the results returned from another column:

```
if { [::dbs::has_redo SOME_FK] == 1 } {
set Var [::dbs::get_redo SOME_FK]
} elseif { [::dbs::has_undo SOME_FK] == 1 } {
set Var [::dbs::get_undo SOME_FK]
}
set a "NULL"
set stream [ ::dbs::create_stream -dst "select SOME_TEXT from
TARGET_TEST_USER.TST_TCL2 where SOME_PK = :1 <char(32)>" ]
::dbs::bind_in $stream "$Var"
::dbs::flush $stream
while { [::dbs::check_eof $stream] == 1 } {
set a [ ::dbs::bind_out $stream ]
::dbs::set_redo TEST $a
}
::dbs::delete_stream $stream
```

In this example:

- SOME_FK is a source table column.
- TEST is a virtual column.
- TARGET_TEST_USER is a target database user.
- TST_TCL2 is a target table for which the user is the owner.

- The following example converts a source string value into a number field:

```
if { [::dbs::has_redo SOME_CHAR] == 1 } {
::dbs::set_redo TEST [::dbs::decimal [::dbs::get_redo SOME_CHAR]]
} elseif { [::dbs::has_undo SOME_CHAR] == 1 } {
::dbs::set_redo TEST [::dbs::decimal [::dbs::get_undo SOME_CHAR]]
}
```

In this example:

- SOME_CHAR is a source table column.
- TEST is a virtual column.

- The following example compares two values to determine if one value is less than, equal to, or greater than another value:

```
if { [::dbs::has_redo SOME_NUMBER] == 1 } {
set Var [::dbs::get_redo SOME_NUMBER]
} elseif { [::dbs::has_undo SOME_NUMBER] == 1 } {
set Var [::dbs::get_undo SOME_NUMBER]
}

if { [::dbs::decimal $Var] == 0 } {
::dbs::set_redo TEST "Equal zero"
} elseif { [::dbs::decimal $Var] < 0 } {
::dbs::set_redo TEST "Less than zero"
```

```
} elseif { [::dbs::decimal $Var] > 0 } {
::dbs::set_redo TEST "Greater than zero"
}
```

In this example:

- SOME_NUMBER is a source table column.
- TEST is a virtual column.

- The following example demonstrates string trimming and capitalizing.

```
if { [::dbs::has_redo SOME_CHAR] == 1 } {
::dbs::set_redo TEST [string toupper [string trim [::dbs::get_redo SOME_CHAR]]]
} elseif { [::dbs::has_undo SOME_CHAR] == 1 } {
::dbs::set_redo TEST [string toupper [string trim [::dbs::get_undo SOME_CHAR]]]
}
```

In this example:

- SOME_CHAR is a source table column.
- TEST is a virtual column.

- The following example replicates the current timestamp value that is stored on the source to the target:

```
set Vart [::dbs::timestamp [clock format [clock seconds] -format "%Y-%m-%d"] [clock
format [clock seconds] -format "%H:%M:%S"] 0]
::dbs::set_redo VTIMESTAMP $Vart
```

In this example, VTIMESTAMP is a virtual column.

The target value includes the source time, the target time, and the difference.

Filtering Change Records

You need to perform some advanced filtering of the change records to be replicated to the target database. This filtering cannot be done from the Data Replication Console.

For this example, create the following filter expression that skips rows with even values in the ID_CALL column:

```
if { [ expr [expr [ ::dbs::get_redo ID_CALL ]] % 2 ] != 0] } {
::dbs::skip
}
```

Then assign the expression to a virtual column in a source table.

Handling Undo and Redo Values in Merge Apply Processing

For Merge Apply processing of deletes and updates, you need to process both undo and redo records. You create an expression that checks for the existence of redo data for the appropriate change record and extracts the undo value if the redo data does not exist.

The following example expression sets the redo and undo values of the VIRT virtual column to the result of the some_proc procedure:

```
proc get_merged_value { in_c } {
if { [ ::dbs::has_redo $in_c ]  } {
return [ ::dbs::get_redo $in_c ]
}  else {
return  [ ::dbs::get_undo $in_c ]
}
}
proc get_undo_only { in_c } {
if { [ ::dbs::has_undo $in_c ] } {
return [ ::dbs::get_undo $in_c ]
} else {
return 0
}
```

```
}
proc some_proc { value } {
return "$value - is processed "
}

::dbs::set_redo VIRT [ some_proc [ get_merged_value $CALL_DATE ]]
::dbs::set_undo VIRT [ some_proc [ get_undo_only $CALL_DATE]]
```

This example populates the undo value of the virtual column by using the undo value of the input column.

Adding Tcl Packages and Third-Party Extensions to a Script

The TCL Script Engine is based on Tcl/Tk 8.5.9 distribution, which comes with the TCLLIB standard library. You can use all packages that the Data Replication distribution provides, without restriction.

To attach packages to a script, use the package require command. Informatica recommends adding packages to a script for the initialization or finalization processing stages.

The following example attaches matrix and csv packages, creates the matrix object, extracts cvs data from the file to the matrix object, and extracts the appropriate cell from the matrix object:

```
# dbs_worker_start.tcl - starting script file
 package require Tcl 8.2
    package require struct::matrix
    package require Tcl 8.3
    package require csv
    package require Tcl 8.2

set file [open ./hash.csv r]
    ::struct::matrix csv_matrix
    ::csv::read2matrix  $file csv_matrix "," auto
set des_maker \
[ csv_matrix get cell 0 \
[ lindex [ lindex [ csv_matrix search all "des_maker" ] 0 ] 1 ] ]
```

Also, you can use third-party Tcl extensions such as Oratcl or the SQLite TCL interface. To install an extension, see the third-party documentation. Use `./support/tcl/` as TCLROOT directory. The following example adds the Oratcl package to a finalization script:

```
package require Oratcl
    package require -exact Oratcl 4.5
set lh [ oralogon some_login/some_pass@//some_host:1521/orcl ]
    puts [orainfo version]
    puts [ orainfo server $lh ]
    set sh [ oraopen $lh ]

set dic [ ::dbs::getglob hash1 ]
    dict for { key value } $dic {
        set sql {
insert into DUMP (COLUMN1,COLUMN2) values ( :key , value )
}
            oraparse $sh $sql
            orabind $sh :key $key :value $value
                    oraexec  $sh
            }
        oracommit $lh
        oraclose  $sh
        oralogoff $lh
```

Using Tcl Scripts to Meet Basic Transformation Requirements

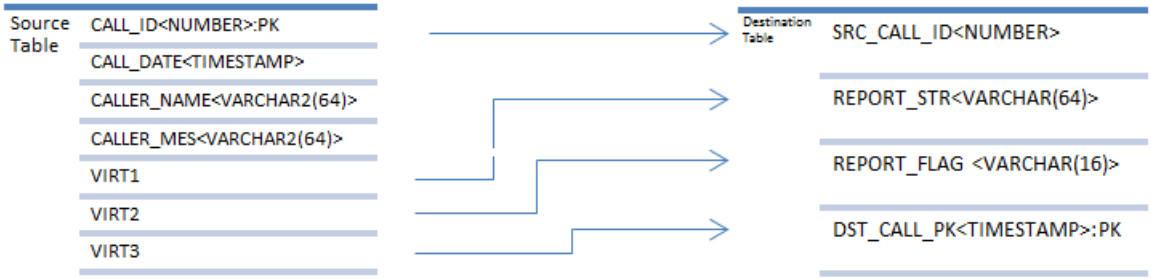
You can use Tcl scripts and the Tcl Script Engine in the transform and load stages of an ETL pipeline.

You can create scripts to selectively load data, convert column data, calculate new values, generate key values, split a column into multiple columns, perform aggregation functions, and apply a transformation.

In this example, you perform the following processing:

1. Filter records to replicate only the records that have a CALLER_NAME column value other than "Mike."
2. Load a Tcl-generated string to a target column named REPORT_STR.
3. Load a Tcl-generated string to a target column named REPORT_FLAG.
4. Load the source CALL_DATE column value to the target DST_CALL_PK column as a primary key.

The following figure shows the resulting replication pattern:



First, define the target primary key or define keys in the database internally. The Tcl Script Engine does not know if a virtual column is mapped to a primary key column. Also, from the Data Replication Console, add the VIRT1, VIRT2, and VIRT3 virtual columns.

Then create the expressions to perform the required functions:

- **Selective loading of change records**. The following expression skips the replication of insert, update, and delete records in which the value of the CALLER_NAME column is not equal to "Mike":

```
if { [expr [ ::dbs::get_redo CALLER_NAME ] != "Mike" ] } {
::dbs::skip
}
```

- **Column datatype conversion and transformation and calculated values**. The following expression calls the user-defined Tcl procedure "get_season" and assigns its return value to the VIRT2 virtual column, which is mapped to REPORT_FLAG target column:

```
set cur_date [ ::dbs::get_redo CALL_DATE ]
set season [ get_season $cur_date]
::dbs::set_redo VIRT1 $season
```

  The get_season procedure gets a datetime object as an input parameter and returns "summer," "autumn," "w inter," or "spring." The procedure performs the conversion and logical transformation of values from datetime to varchar.

- **Data aggregation**. The following expression builds a report string from source values and assigns the string to the VIRT2 virtual column:

```
set cur_date [ ::dbs::get_redo CALL_DATE ]
set season [ get_season $cur_date]
    set caller [ ::dbs::get_redo CALLER_NAME ]
    set msg "The call $id was made in $cur_date by $caller. The season is $season"
::dbs::set_redo VIRT2 $msg
```

- **Build a primary key value**. The following expression extracts the CALL_DATE value and assigns it to the VIRT3 virtual column, which is mapped to DST_CALL_PK target column:

```
::dbs::set_redo VIRT3 [ ::dbs::get_undo CALL_DATE ]
```

CHAPTER 3

SQL Scripting

This chapter includes the following topics:

- Using SQL Expressions, 32
- Data Access Operators, 35
- SQL Script Examples, 39

Using SQL Expressions

Data Replication can use SQL expressions that you assign to virtual columns in a source database to calculate values in the target database during InitialSync and Apply processing.

Execution of SQL Expressions

The SQL Script Engine is an SQL expression parser that embeds the SQL language elements that you specify into full SQL statements that InitialSync or Applier tasks use to load data to a target database.

When an InitialSync or Applier task replicates virtual columns with assigned SQL expressions to a target database, the target database executes the SQL expressions and applies calculated values to the mapped columns.

The Applier can process SQL expressions in SQL Apply and Merge Apply modes. Data Replication does not support SQL expressions for targets that use Audit Apply mode, such as Apache Kafka, Cloudera, Hortonworks, and Flat File targets.

InitialSync can process SQL expressions when loading data to a target with an appropriate native load utility, ODBC driver, or database API.

For InitialSync processing of SQL expressions with a native load utility and for the Applier processing of SQL expressions in Merge Apply mode, Data Replication uses the audit log tables on the target. With the audit log tables, InitialSync and the Applier load data to the target in two stages:

- During the first stage, InitialSync and the Applier load the source data to the audit log tables in bulk.
- During the second stage, InitialSync and the Applier use the before and after image columns in the audit log tables to evaluate the SQL expressions and update data in the audit log tables. InitialSync and the Applier then use SELECT and INSERT statements to transfer data from the audit log tables to the regular target tables.

Generating Audit Log Tables for Processing SQL Expressions

If you run InitialSync with a native load utility, or if you run the Applier in Merge Apply mode, you must generate audit log tables based on mapped target tables to correctly process SQL expressions.

The Applier requires an audit log table for each mapped target table. When you use InitialSync with a native load utility, InitialSync requires an audit log table for each target table that is mapped to a source table for which you defined SQL expressions.

If you map a target table to a source table with a SQL expression, the audit log table for the target table includes the following types of columns:

- Metadata columns.
- Before-image and after-image columns for the mapped target columns. The names of these columns are composed of the target column names followed by the suffix that you defined for the before or after image.
- Before-image and after-image columns for the source columns that you access in the SQL expression but did not map to the target columns. The names of these columns are composed of the source column names preceded by a prefix that is generated for the before or after image based on the suffix.

Before you generate the audit log tables, map all of the virtual source columns on the **Map Columns** tab.

1. Optionally, edit settings on the **Runtime Settings** tab > **Calculated Columns** view to customize audit log tables.

   You can specify the following settings:

   - The audit log table suffix. By default, Data Replication uses _LOG as the suffix.
   - Metadata column names.
   - Before-image and after-image suffixes and prefixes.
     You can specify only the suffixes for the audit log table. Data Replication generates prefixes based on the suffixes. To create a prefix from a suffix, Data Replication removes the leading underscore character, if any, and appends the underscore character to the end of the suffix value.

     By default, Data Replication uses the suffixes _OLD and _NEW and the prefixes OLD_ and NEW_.
   - The target schema that includes the audit log tables. By default, InitialSync and the Applier look for the audit log tables in the schema that includes the regular target tables.
2. Click the **Generate Schema for the Target** button on the toolbar.

   The **Generating Schema for the Target** dialog box appears.
3. Under **Source/Target**, select **Target** to generate audit log tables based on regular target tables.
4. In the **Schema/owner** field, enter the name of the schema or owner that includes the regular target tables.
5. Under **Tables**, select the target tables for which to generate audit log tables.
6. Under **Target**, in the **Schema** field, enter the name of the schema or owner to use for generating the audit log tables.

   Verify that the specified schema or database matches the schema or database on the **Runtime Settings** tab > **Calculated Columns** view.
7. In the **Database** type list, select the target database type.
8. Select **Generate audit log tables**.
9. In the **Audit tables suffix** field, enter a suffix for the audit log tables.

   Verify that the specified suffix matches the audit log table suffix on the **Runtime Settings** tab > **Calculated Columns** view.

10. Perform one of the following actions:
    - Click **Convert** to generate SQL CREATE TABLE scripts.
    - Click **Execute** to create the audit log tables on the target immediately.

Syntax of SQL Expressions

Data Replication inserts SQL expressions into SQL statements that run on the target database to replicate data. Ensure that the SQL expressions conform to the syntax of the target database.

For example, the standard SQL syntax for concatenation is to use the || operator. However, if you have a Microsoft SQL Server target database, use the + operator. If you have a MySQL target database, use the CONCAT function.

Important: Data Replication cannot process SQL expressions that use the :: construct for datatype casting. Instead, you must use explicit CAST or CONVERT requests.

Getting Access to Transactional Data

The SQL Script Engine provides data access operators to get the transactional data that the Extractor captures. These operators work with all target databases.

You can use these operators to get the following data:

- REDO values
- UNDO values
- Metadata

WHERE Clauses in SQL Expressions

You must add a WHERE clause to an SQL expression if you include the virtual column with the SQL expression in the virtual index of a source table. The SQL Script Engine uses the SQL expression that you enter in the **WHERE clause** field to uniquely identify rows when replicating Update and Delete operations.

Note: You can add a WHERE clause to a SQL expression in the **SQL Expression Editor** window.

In most cases, you just copy the SQL expression that you specify for the Insert operation and replace the :NEW.*Column_Name* operators with the :OLD.*Column_Name* operators.

In Audit Apply mode, an SQL expression in the **WHERE clause** field calculates a value for the *virtual_column_name*_OLD column in the target audit table for Update and Delete operations.

Validating SQL Expressions

After you map a virtual source column to a target column, you can validate the SQL expression that is assigned to the virtual column in the target database.

1. On the **Map Tables** tab, right-click the mapped source table row for which you defined the SQL expression and click **Add/Show Table Expression**.

   The **Add/Show expressions** dialog box appears.
2. Select the expression row and click **Edit**.
3. In the **Insert a SQL Expression** dialog box, click **Verify**.

Data Replication validates the SQL expression in the target database.

- If the target database can execute the SQL expression for the selected column, the Data Replication Console displays a message that indicates all functions are valid.
- If the target database cannot execute the SQL expression for the selected column, the Data Replication Console displays a database error message.

4. Click **OK** to close the message.

Data Access Operators

The SQL Script Engine provides data access operators to get the transactional data that the Extractor captures.

Access to REDO Values

:NEW.*<column_name>*

Returns the REDO value of the *<column_name>* column in a source table.

The SQL Script Engine binds fixed-point number datatypes as text. If you perform arithmetic operations on REDO values that have number datatypes in the SQL expression, ensure that the syntax of the target database allows these operations on numbers that are represented as text. If the syntax of the target database requires numeric operands, convert the REDO values to the appropriate number datatypes in the SQL expression. For example, for a SQL Server target, use the following syntax to multiply two integers:

```
convert(int, :NEW.COLUMN1) * convert(int, :NEW.COLUMN2)
```

:NEW.*<column_name>*.is_available

For Applier processing in SQL Apply mode, returns 1 if the *<column_name>* column in a source table has the REDO value. Returns 0 if the *<column_name>* column in a source table has no REDO value.

For Applier processing in Merge Apply and Audit Apply modes, returns 1 if the value of the *<column_name>*_NEW column in the audit log table is not a null value. Returns 0 if the *<column_name>*_NEW column has a null value.

The *<column_name>*_NEW columns in audit log tables contain null values in the following cases:

- The source column has a null value.
- For Oracle sources, the source row is updated but the specified source column is not changed, and the column is not included into a supplemental log group.
- The source row is deleted.

For InitialSync processing, returns a null value.

:NEW.*<column_name>*.is_null

For Applier processing in SQL Apply mode, returns 1 if the REDO value of the *<column_name>* column in a source table is a null value. Returns 0 if the REDO value of the *<column_name>* column in a source table is not a null value.

For Applier processing in Merge Apply and Audit Apply modes, returns 1 if the *<column_name>*_NEW column in the audit log table contains a null value. Returns 0 if the *<column_name>*_NEW column is not a null value.

Tip: If the source column has no REDO value, the :NEW.*<column_name>*.is_null operator returns 1. In SQL Apply mode, you can use the :NEW.*<column_name>*.is_available operator to determine if the source column has a REDO value. To do so, enter the :NEW.*<column_name>*.is_available operator before the :NEW.*<column_name>*.is_null operator in the script. The following SQL expression determines whether the COL3 column has a REDO or UNDO value and then compares this value to the null value:

```
CASE WHEN :NEW."COL3".IS_AVAILABLE  = 1 THEN :NEW."COL3".IS_NULL
WHEN  :OLD."COL3".IS_AVAILABLE = 1 THEN :OLD."COL3".IS_NULL
ELSE not_in_supplemental_group
END
```

If the column has neither a REDO nor UNDO value, the expression returns the *not_in_supplemental_group* value.

For InitialSync processing, returns a null value.

:NEW.*<column_name>*.type

Returns a VARCHAR string that contains the datatype of the *<column_name>* column in the source table.

Access to UNDO Values

:OLD.*<column_name>*

For Applier processing, returns the UNDO value of the *<column_name>* column in a source table.

The SQL Script Engine binds fixed-point number datatypes as text. If you perform arithmetic operations on UNDO values that have number datatypes in the SQL expression, ensure that the syntax of the target database allows these operations on numbers that are represented as text. If the syntax of the target database requires numeric operands, convert the UNDO values to the appropriate number datatypes in the SQL expression. For example, for a SQL Server target, use the following syntax to multiply two integers:

```
convert(int, :OLD.COLUMN1) * convert(int, :OLD.COLUMN2)
```

For InitialSync processing, returns a null value.

:OLD.*<column_name>*.is_available

For Applier processing in SQL Apply mode, returns 1 if the *<column_name>* column in a source table has the UNDO value. Returns 0 if the *<column_name>* column in a source table has no UNDO value.

For Applier processing in Merge Apply and Audit Apply modes, returns 1 if the value of the *<column_name>*_OLD column in the audit log table is not a null value. Returns 0 if the *<column_name>*_OLD column has a null value.

The *<column_name>*_OLD columns in audit log tables contain null values in the following cases:

- The source row is inserted.
- For Oracle sources, the source row is updated but the specified source column is not changed, and the column is not included into a supplemental log group.
- The source column that contains a null value is updated.

For InitialSync processing, returns a null value.

:OLD.*<column_name>*.is_null

For Applier processing in SQL Apply mode, returns 1 if the UNDO value of the *<column_name>* column in a source table is a null value. Returns 0 if the UNDO value of the *<column_name>* column in a source table is not a null value.

For Applier processing in Merge Apply and Audit Apply modes, returns 1 if the *<column_name>*_OLD column in the audit log table contains a null value. Returns 0 if the *<column_name>*_OLD column is not a null value.

Tip: In SQL Apply mode, you can use the :OLD.*<column_name>*.is_available operator to determine if the source column has an UNDO value. To do so, enter the :OLD.*<column_name>*.is_available operator before the :OLD.*<column_name>*.is_null operator in the script. If the source column has no UNDO value, the :OLD.*<column_name>*.is_null operator returns 1.

For InitialSync processing, returns a null value.

:OLD.*<column_name>*.type

For Applier processing, returns a VARCHAR string that contains the datatype of the *<column_name>* column in a source table.

For InitialSync processing, returns a null value.

Access to Metadata

:META.REC.NUM_IN_TX

For Applier processing, returns the sequence number of a DML operation in a transaction.

For InitialSync processing, returns a null value.

:META.REC.OPTYPE

Returns a string that contains the DML operation type. For Applier processing, the operation type must be one of the following values:

- **I** for Inserts.
- **U** for Updates.
- **D** for Deletes.

For InitialSync processing, the operation type must be **O** for Inserts.

:META.REC.OBJID

For Applier processing, returns the integer value of OBJECT ID of the source table.

For InitialSync processing, returns a null value.

:META.REC.OBJ_DATA_ID

For Applier processing, returns the integer value of DATA OBJECT ID.

For InitialSync processing, returns a null value.

:META.REC.OP_DT

For Applier processing, returns the DATETIME object that contains the creation time of a record.

For InitialSync processing, returns a null value.

:META.REC.SCN

For Applier processing, returns a string that contains the SCN value for an added source record.

For InitialSync processing, returns a null value.

:META.TRANS.BEGIN_DT

For Applier processing, returns a string that contains the DATETIME value for opening a transaction in the source database.

For InitialSync processing, returns a null value.

:META.TRANS.BEGIN_SCN

For Applier processing, returns a string that contains the SCN value for opening a transaction in the source database.

For InitialSync processing, returns a null value.

:META.TRANS.END_DT

For Applier processing, returns a string that contains the DATETIME value for closing a transaction in the source database.

For InitialSync processing, returns a null value.

:META.TRANS.END_SCN

For Applier processing, returns a string that contains the SCN value for closing a transaction in the source database.

For InitialSync processing, returns a null value.

:META.TRANS.ID

For Applier processing, returns a string that contains the transaction ID in the source database.

For InitialSync processing, returns a null value.

:META.TRANS.STATE

For Applier processing, returns a string with a transaction state. Valid values are:

- **R** for rollback
- **A** for activated
- **C** for commit

For InitialSync processing, returns a null value.

:META.TRANS.USER_ID

For Applier processing, returns the user ID of the database user who performed a DML operation.

For Applier processing of change data from DB2 sources, returns a null value.

For InitialSync processing, returns a null value.

:META.TRANS.USER_NAME

For Applier processing, returns the name of the database user who performed a DML operation.

Note: If the transaction logs do not contain the user name, returns a null value.

For InitialSync processing, returns a null value.

Metadata Operators in Audit and Merge Apply Modes

In Audit Apply and Merge Apply modes, you can access only metadata in the audit tables. The following operators are available in these modes:

- :META.REC.OPTYPE
- :META.REC.OP_DT
- :META.TRANS.ID
- :META.TRANS.END_SCN
- :META.TRANS.END_DT

- :META.REC.NUM_IN_TX
- :META.TRANS.USER_ID
- :META.TRANS.USER_NAME

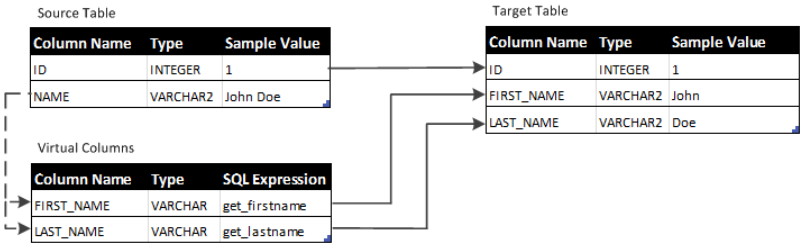
SQL Script Examples

To get started, review some simple SQL expressions.

String Extraction Example

This example shows how to split the value of the NAME source column into the FIRST_NAME and LAST_NAME target columns.

The following figure shows how the source table columns and virtual columns map to the target table:



Oracle does not have a system function to split a string. You must add the following stored function, split_text, to the target database:

```
create or replace function target_user.split_text ( some_text varchar2, word_pos int)
return varchar2
as
result varchar2(2000) default null;
begin
select name into result from (
select trim(regexp_substr(some_text,'[^ ]+',1,level)) name, level lv
from dual
CONNECT BY
regexp_substr(some_text,'[^ ]+',1,level) is not null
)
where lv=word_pos;
return result;
end;
```

The following SQL expressions for Oracle targets call the split_text stored function to get the first and second words from the space-separated value of the NAME column:

get_firstname

```
split_text(:NEW.NAME, 1)
```

get_lastname

```
split_text(:NEW.NAME, 2)
```

For SQL Server targets, we can split the first and the last name from the space-separated value of the NAME column without calling a stored function:

get_firstname

```
LEFT(:NEW.FULL_NAME, CHARINDEX(' ', :NEW.FULL_NAME) - 1)
```

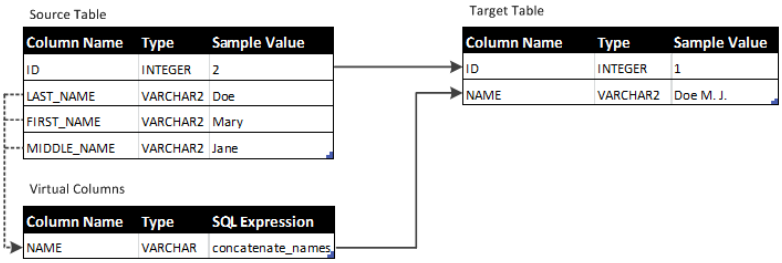
get_lastname

```
RIGHT(:NEW.FULL_NAME, CHARINDEX(' ', REVERSE(:NEW.FULL_NAME)) - 1)
```

String Concatenation Example

This example shows how to concatenate values of the FIRST_NAME, LAST_NAME, and MIDDLE_NAME columns in a source database to write a single name value into the NAME column in a target database.

The following figure shows how the source table columns and virtual column map to the target table columns:



Use the concatenate_names SQL expression for Oracle targets to concatenate the values of the LAST_NAME, FIRST_NAME, and MIDDLE_NAME columns:

```
:NEW.LAST_NAME||' '||:NEW.FIRST_NAME||' '||:NEW.MIDDLE_NAME
```

The concatenate_names SQL expression for SQL Server targets:

```
:NEW.LAST_NAME+' '+:NEW.FIRST_NAME+' '+:NEW.MIDDLE_NAME
```

In the following example, the concatenate_names expression is modified to get the LAST_NAME value and to use the Oracle SUBSTR system procedure to append the first letter of the FIRST_NAME and MIDDLE_NAME column values:

```
:NEW.LAST_NAME||' '||SUBSTR(:NEW.FIRST_NAME,1,1)||'. '||SUBSTR(:NEW.MIDDLE_NAME,1,1)||'.'
```

The same script for SQL Server targets:

```
:NEW.LAST_NAME+' '+SUBSTRING(:NEW.FIRST_NAME,1,1)+'. '+SUBSTRING(:NEW.MIDDLE_NAME,
1,1)+'.'
```

Another version of the concatenate_names SQL expression for Oracle targets checks if the REDO value of the MIDDLE_NAME column is null. If the MIDDLE_NAME is not null, the expression uses the Oracle SUBSTR system procedure to append the first letter of the FIRST_NAME and MIDDLE_NAME column values to the LAST_NAME value. If the MIDDLE_NAME value is null, the expression appends only the first letter of the FIRST_NAME column value to the LAST_NAME value.

```
CASE :NEW.MIDDLE_NAME.IS_NULL WHEN 0  THEN
:NEW.LAST_NAME||' '||SUBSTR(:NEW.FIRST_NAME,1,1)||'. '||SUBSTR(:NEW.MIDDLE_NAME,1,1)||'.'
ELSE :NEW.LAST_NAME||' '||SUBSTR(:NEW.FIRST_NAME,1,1)||'.' END
```

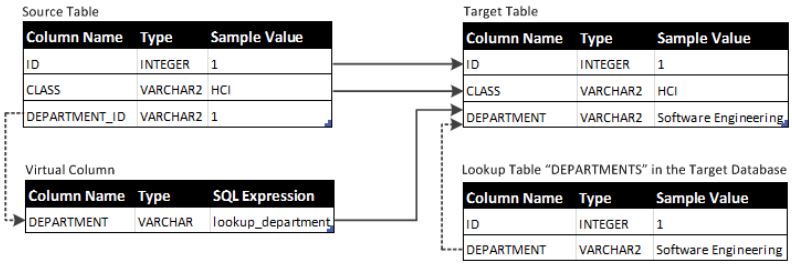
The same script for SQL Server targets:

```
CASE :NEW.MIDDLE_NAME.IS_NULL WHEN 0 THEN :NEW.LAST_NAME+' '+SUBSTRING(:NEW.FIRST_NAME,
1,1)+'. '+SUBSTRING(:NEW.MIDDLE_NAME,1,1)+'.'
ELSE :NEW.LAST_NAME+' '+SUBSTRING(:NEW.FIRST_NAME,1,1)+'.' END
```

Lookup Example

This example shows how to look up values from a table in the target database and replicate query results to the target table of a replication.

The following figure shows how the source table columns and virtual column map to the target table columns:



The lookup_department SQL expression retrieves a department name from a lookup table in the target database by using the REDO value of the DEPARTMENT_ID column in the source table:

```
(SELECT DEPARTMENT FROM DEPARTMENTS WHERE ID=:NEW.DEPARTMENT_ID)
```

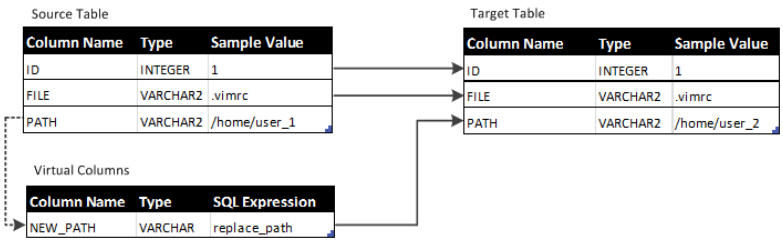
The result of the query is replicated to the DEPARTMENT column in the target table.

Important: The lookup table must be in the target database because the SQL Script Engine executes SQL expressions on the target.

String Substitution Example

This example shows how to use the SQL Script Engine to substitute a string value.

The following figure shows how the source table columns and virtual column map to the target table columns:



The replace_path SQL expression gets the REDO value of the PATH column in a source table and replaces the string '/home/user_1' in the REDO value with '/home/user_2':

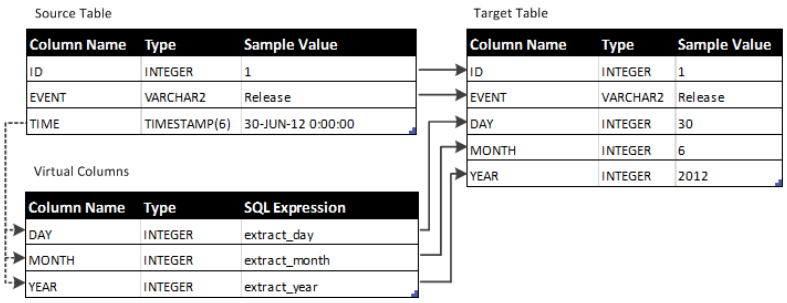
```
REPLACE(:NEW.PATH, '/home/user_1', '/home/user_2')
```

Note: This SQL expression is identical for Oracle and SQL Server targets.

Date Manipulation Example

This example shows how to manipulate datetime values.

The following figure shows how the source table columns and virtual columns map to the target table columns:



The following SQL expressions use the Oracle EXTRACT function to get day, month, and year values from the TIME column:

extract_day

```
EXTRACT (DAY FROM :NEW.TIME)
```

extract_month

```
EXTRACT (MONTH FROM :NEW.TIME)
```

extract_year

```
EXTRACT (YEAR FROM :NEW.TIME)
```

The following SQL expressions get day, month, and year values from the TIME column for SQL Server targets:

extract_day

```
DATEPART (dd,:NEW.TIME)
```

extract_month

```
DATEPART (mm,:NEW.TIME)
```

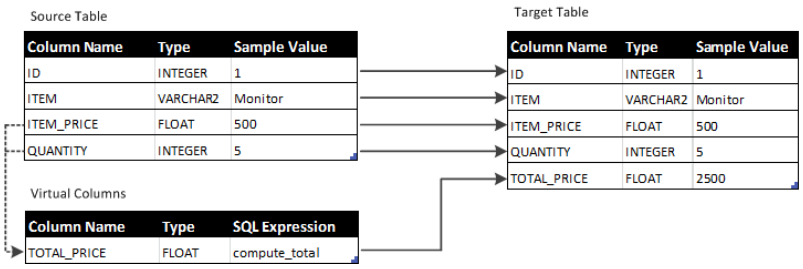
extract_year

```
DATEPART (yyyy,:NEW.TIME)
```

Computing Example

This example shows how to multiply the values of two source columns and replicate the result to a new target column.

The following figure shows how the source table columns and virtual column map to the target table columns:



Use the following compute_total SQL expression:

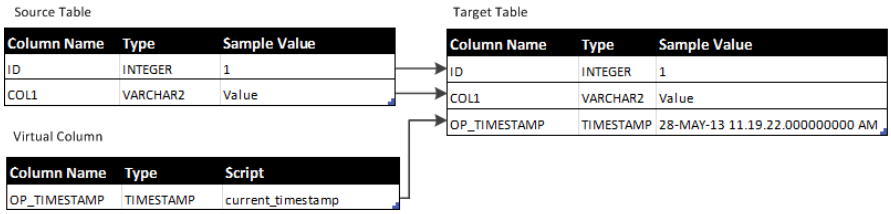
```
:NEW.ITEM_PRICE*:NEW.QUANTITY
```

In this example, the compute_total SQL expression is assigned to the TOTAL_PRICE virtual column in the source table. The expression multiplies the REDO values of the ITEM_PRICE and QUANTITY columns. The result is replicated to the TOTAL_PRICE column in the target database.

Current Timestamp Example

This example shows how to add the current date and time of the target operating system when applying data to the target table. Also, this example shows how to get the creation time stamp of the source change record when applying data.

The following figure shows how the source table columns and virtual column map to the target table columns:



The current_timestamp SQL expression is assigned to the OP_TIMESTAMP virtual column in the source table. The expression gets the current date and time of the target operating system when the Applier applies change data to the target table.

- For Oracle targets, use the following current_timestamp SQL expression:

  ```
  SYSDATE
  ```

- For Microsoft SQL Server targets, use the following current_timestamp SQL expression:

  ```
  GETDATE()
  ```

Alternatively, the following version of the current_timestamp SQL expression accesses metadata to get the creation time of a record on the source system during apply processing:

```
:META.REC.OP_DT
```

Note: This SQL expression is identical for all target database types.

For InitialSync processing, this expression returns a null value.

INDEX

A

audit log tables
  generating for processing SQL expressions 33

C

commands
  ::dbs::repo global variable 23
  Applier thread ID and interpreter ID 23
  bind string to SQL statement stream command 25
  change record and transaction metadata 23
  check end of SQL statement stream command 25
  create SQL statement stream command 25
  datatype compare command 22
  datatype object commands 19
  datetime or timestamp extract command 22
  delete SQL statement stream command 25
  flush SQL statement stream command 25
  get global context variable command 21
  I/O commands 18
  lock global context command 21
  log command 21
  set global context variable command 21
  skip current change record command 21
  stop Applier command 21
  Tcl Script Engine interaction commands 21
  threaded context commands 21
  unbind string from SQL statement stream command 25
  unlock global context command 21

E

execution of script
  controlling 18

G

global context
  setting variables for interpreters 17

I

input data
  methods of accessing 14

S

source columns
  defining a virtual index 12
SQL examples
  adding the current timestamp of the operation 43
  computing values for a target column 42
  concatenating strings 40
  extracting words from a string 39
  looking up values in a target table 41
  manipulating dates 42
  substituting strings 41
SQL expressions
  defining 9
  execution 32
  syntax 34
  validating in the target database 34
  WHERE clauses 34
SQL operators
  access to metadata 37
  access to REDO values 35
  access to UNDO values 36

T

Tcl examples
  adding Tcl packages and third-party extensions to a script 30
  filtering of change records 29
  handling undo and redo values in Merge Apply processing 29
  using Tcl scripts to meet ETL requirements 30
Tcl expressions
  defining 8
Tcl Script Engine
  processing stages 15

V

virtual columns
  assigning Tcl or SQL expressions 10
  setting column values 15